

Nodges – Projektbericht (GPT-5 Edition)

1. Zusammenfassung

Nodges ist ein 3D-Visualisierungstool für Netzwerke auf Basis von Three.js (WebGL, ES6 Modules). Es bietet Import/Export, Analyse, Interaktion (Raycasting, Auswahl, Highlighting), Layout- und State-Management sowie verschiedene Hilfs-Manager (Leistung, Suche, Pfade, Gruppen) und Worker-unterstützte Layouts. Der Code ist modular aufgebaut (Core-, Utils-, Effects-, Objects-Schichten) und zielt auf große Netzwerke (Instancing, Optimierungen) ab.

2. Projektstruktur und Artefakte

- Hauptartefakte: index.html, main.js, colorscheme.css, favicon.
- Paketmanagement: package.json (three ^0.161.0, ES Module).
- Datenbeispiele: data/*.json (small/medium/large/mega, US-Systeme, Ikosaeder, Future-Formatbeispiele).
- Quellcode:
 - objects/: 3D-Objektrepräsentationen (Node, Edge, NodeMeshFactory, nodeObjects).
 - src/core/: zentrale Manager (State, UI, Layout, Interaction, CentralEvent).
 - src/utils/: umfangreiche Utility-Manager (Import/Export/File, FutureData*, Raycast, Search, Selection, LayoutGUI, Analyzer, PathFinder, Performance, BatchOps, EdgeLabel, Ground, KeyboardShortcuts, Groups, Neighborhood, ConnectInterfaceAdapter).
 - src/effects/: GlowEffect, HighlightManager.
 - src/workers/: layout-worker.js.
- Doku: README.md (Kurz), doc/definition_json.txt, gemniDoc/*.mermaid, gemniDoc/dokumentation_gesamt.pdf.

3. Hauptkomponenten (Überblick)

- Core
 - StateManager: Applikationszustand (Selektion, Filter, Modi, Layer-Flags, UI-Zustand...).
 - UIManager: UI-Bindungen, Panels, Hotkeys/Overlays, Interop mit anderen Managern.
 - LayoutManager: Layout-Algorithmen/Worker-Orchestrierung, Positionsupdates, Re-Layout.
 - InteractionManager: Maus/Tastatur, Raycasting, Auswahl, Drag/Drop, Multi-Select.
 - CentralEventManager: Event-Hub (Publish/Subscribe, Delegation, cross-manager Kommunikation).
- Utils
 - ImportManager/ExportManager/FileHandler: Datei-IO, JSON-Formate, Speichern/Laden.
 - FutureDataParser/Manager/Validator: neues Datenformat, Validierung, Migrationspfade.
 - RaycastManager: effizientes Picking, InstancedMesh-Unterstützung.
 - SearchManager/SelectionManager/BatchOperations: Suchen, Auswahl, Massenoperationen, Undo-Basis.
 - LayoutGUI/NetworkAnalyzer/PathFinder: Layoutsteuerung, Graph-Analysen, Pfadsuche.
 - PerformanceOptimizer: Caching, Throttling, Rendering-Optimierungen.
 - NodeGroupManager/NeighborhoodHighlighter: Gruppen, Nachbarschaftsmarkierung.
 - EdgeLabelManager: Kanten-Beschriftungen.

- GroundManager: Bodenebene/Ästhetik.
- KeyboardShortcuts: F1-Hilfe und Shortcuts.
- ConnectInterfaceAdapter: Integrations-Adapter/Brücke für externe Schnittstellen.
- Effects
 - GlowEffect: Emissive-/Farb-Intensitätssteuerung für Node/Edge.
 - HighlightManager: Hover-/Selection-Highlights, orchestriert Effektlogik.
- Objects
 - Node: Datencontainer je Knoten.
 - Edge: Kurvige TubeGeometry mit kontrolliertem Bogen (behandelt Mehrkantenfälle, Materialfarbe, userData).
 - NodeMeshFactory: Mesh-Erzeugung (Sphere/Cube/Cone) je Node-Attributen.

4. Datenfluss (vereinfacht)

1. Import (User File/Beispiel) → FileHandler/ImportManager.
2. Optional: FutureDataParser/Validator → FutureDataManager.
3. Erzeugung von Node/Edge-Objekten → NodeMeshFactory, Edge.
4. Scene-Setup (Three.js) → Rendering-Loop.
5. Interaction (RaycastManager) → SelectionManager/HighlightManager → StateManager.
6. LayoutManager (ggf. Worker) → Positionsupdates für Node/Edge → Render.
7. ExportManager → Dateiablage.

5. Interaktionsfluss (Auswahl)

- UI/Event → InteractionManager (Mausbewegung/-klick) → RaycastManager.pick()
- Treffer → SelectionManager.updateSelection() → HighlightManager/GlowEffect anpassen
- StateManager aktualisiert Selektion/Modus → UIManager reflektiert

6. Rendering & Performance

- Three.js Szene mit InstancedMesh für Nodes, TubeGeometry für Edges.
- Raycasting throttled (z. B. 50ms in RaycastManager), letzte Intersection zwischengespeichert.
- PerformanceOptimizer: Reduktion hoher Kosten (Batch-Updates, Sichtbarkeiten, ggf. LOD/Frustum-Ansätze).

7. Architekturprinzipien

- Trennung von Concerns: Core (Koordination), Utils (Funktionen), Effects (Darstellung), Objects (Modelle).
- Event-Driven: CentralEventManager als Hub, lose Kopplung.
- Erweiterbarkeit: Neue Layouts, Parser, Effekte, Analyzer als eigenständige Module.
- Testbarkeit: Manager sind modulare Klassen mit klaren Schnittstellen.

8. Datenmodell (implizit)

- Node: { x, y, z, name, color, size, metadata }
- Edge: geometrische Ableitung aus Start/Ende, Optionen: { color, totalEdges, index, name }
- FutureData: validier-/parsebares Ziel-Format; Transformationslogik in FutureData*.

9. Bekannte Features und Shortcuts

- Multi-Select (Ctrl + Click), Box-Select (Shift + Drag), Ctrl + A, Escape, Delete.
- Navigation (Orbit/Pan/Zoom), F1-Hilfe-Overlay.
- Geplante bzw. teils vorhandene: Export/Import via Shortcuts, Batch Undo.

10. Qualität, Risiken, Empfehlungen

- Qualität: Struktur klar, Module zahlreich, teils lange Dateien (Refactor-Potenzial in Core/Utils).
- Risiken: Fehlende npm Scripts/Dev-Server; nicht alle Schnittstellen sind strikt typisiert; lange Manager-Dateien erhöhen Kopplung.
- Empfehlungen:
 - npm scripts: dev (vite/serve), build (rollup/webpack), lint/test.
 - Striktere Interfaces/Types (JSDoc/TS) für Module, vor allem Events und Daten.
 - Unit-Tests für Parser/Validator/Selection/Pathfinding.
 - Event-Dokumentation und -Kontrakte im CentralEventManager.
 - Edge- und Node-Instancing-Strategie dokumentieren (Skalierung, Limits, LOD).

11. How-To Run (Vorschlag)

- Einfach: `npx serve .` oder `python -m http.server` (statischer Server).
- Optional npm script: "dev": "`npx http-server -p 5173`".

12. Diagramme (extern)

Die Diagramme sind ausgelagert in den Dateien unter `diagrams_gpt5/` und werden extern gerendert. Beim Rendern ist sicherzustellen, dass die Schriftgröße auf A4 nicht unter 11pt liegt. In den .mmd-Dateien ist eine Mermaid-Init-Direktive mit `fontSize: '15px'` hinterlegt, um dies sicherzustellen.

Siehe nummerierte Referenzen [3.1]–[3.9].

13. Referenzen (nummeriert)

- [1] bericht_gpt5.md – Dieses Dokument
- [2] bericht_architektur_gpt5.md – Architekturbericht
- [3.1] diagrams_gpt5/architecture_overview.mmd – Architekturübersicht
- [3.2] diagrams_gpt5/module_dependency.mmd – Modulabhängigkeiten
- [3.3] diagrams_gpt5/data_flow.mmd – Datenfluss
- [3.4] diagrams_gpt5/event_sequence_selection.mmd – Sequenzdiagramm: Selection
- [3.5] diagrams_gpt5/import_pipeline.mmd – Flowchart: Import-Pipeline
- [3.6] diagrams_gpt5/state_machine_selection.mmd – Zustandsautomat: Selection
- [3.7] diagrams_gpt5/mindmap_system.mmd – Mindmap: System Overview
- [3.8] diagrams_gpt5/flowchart_startup.mmd – Flowchart: App-Startup
- [3.9] diagrams_gpt5/class_map_core.mmd – Klassendiagramm: Core & Effects

Ende des Projektberichts.